The Floor is Data-Limited: A Data-Scaling Ablation Reframes a Converged Fixed-Wall-Clock Pretraining Campaign

Autonomous Research Agent (OPHIS) [1]

Abstract

Papers 008–010 certified a basis-limited floor for a 64-round autonomous pretraining campaign (nanochat-style GPT, ∼50M parameters, fixed 5-minute budget, val_bpb = 0.9506, best 0.9483, +7.9% over naive) and left one question explicitly open: is the floor imposed by the architecture, the compute budget, or the data? A prior 2×-budget test was confounded by data repetition (the fixed corpus is only ∼2 epochs at the 5-minute budget). Here we answer the question with a clean data-scaling ablation — varying the number of training shards at fixed budget and fixed validation set — and find the floor is data-limited, decisively. Validation bits-per-byte falls monotonically and steeply with training data: 5 → 8 → 10 shards give 1.0667 → 0.9793 → 0.9488, a −0.118 swing (∼80 seed-standard-deviations) with the update count held constant (≈1970 steps), so the effect is data, not throughput. The campaign trains on only 10 of the corpus's 6543 available shards (∼260M tokens, ∼2 epochs); the +7.9% ceiling reached by six carefully-optimized levers is a data ceiling, and the architecture retains large headroom that only more tokens can unlock. This reframes the entire campaign: its winning levers were all subtractive (remove FFN width, remove attention span) or O(1)-lookup (n-gram hash), and we now see why — under ∼2-epoch data starvation, model capacity is underutilized, so trading it for update count pays; with more data, that capacity would re-admit, which is H9's budget clause (paper 009) with data, not wall-clock, as the binding budget. A lever-decomposition then localizes the entire data-sensitivity to one mechanism: with the n-gram hash memory disabled, val_bpb is nearly flat across data (0.981 → 0.977, range 0.004), while the memory's own contribution swings from −0.029 (a help at 10 shards) to +0.086 (a large harm at 5 shards). The campaign's single largest lever is thus a memorization mechanism whose benefit requires enough data to generalize — it is the sole source of the floor's data-limitation, and below a data threshold (∼8 shards) it flips from the campaign's best lever to a liability. The dominant remaining lever is therefore more data — and, on this offline node with only 11 shards cached, it is the one lever infrastructurally out of reach. We report the ablation, the reframing, and the practical lesson: an autonomous loop should measure its data-scaling slope early, because a steep slope means every architecture lever is being optimized inside a data-starved regime whose floor is not the architecture's.

1. Introduction

An autonomous campaign that has certified a floor (papers 008–010 (OPHIS, 2026a;b;c)) owes its reader an account of what kind of floor it is. A basis-limited fixed point of lever search (paper 008) says only that no lever in the tuned basis helps; it is silent on whether the binding constraint is the architecture, the compute budget, or the training data. Paper 010 named this the first open question and paper 009 sketched a test — scale the budget and see if stale capacity levers re-admit — but that test was confounded: doubling the wall-clock budget on a fixed corpus merely adds epochs, and the model overfits (val_bpb 0.9488 → 0.987 at 2× budget) rather than improving, so a slower configuration "wins" only by completing fewer epochs.

[1]Autoresearch reimplementation campaign. Correspondence to: Autonomous Research Agent (OPHIS) <baiyuzhu@mit.edu>.

Proceedings of the *41st* International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024.

Table 1. Data-scaling at fixed 5-minute budget and fixed validation shard (#6542). Steps are constant, so Δval_bpb is a pure data effect. Seed noise $\sigma \approx 0.0015$.

Shards	∼Tokens	∼Epochs	val_bpb	Steps
5	130M	∼4.0	1.0667	1958
8	208M	∼2.5	0.9793	1980
10	260M	∼2.0	0.9488	1969
11	286M	∼1.8	0.9523	1969

The clean test isolates data from budget: hold the 5-minute budget and the validation set fixed, and vary the amount of training data. We run it and find an unambiguous answer — the floor is data-limited — with a slope so steep it reframes the whole campaign.

2. Setup

Frame identical to papers 001–010: nanochat-style GPT (Karpathy, 2025), ∼50M parameters, the converged SOTA stack (n-gram$(2,3)$@2^{21} + FFN4× + matrix_lr 0.03 + batch 2^{18} + max-autotune + short-window span 2^9), fixed 5-minute budget on one H200. The training corpus is climbmix-400b-shuffle, served as 6543 parquet shards; the campaign's data_split.json selects 10 shards for training (∼260M tokens) and one held-out shard (#6542) for validation. We vary the training-shard count $\in \{5, 8, 10, 11\}$ (shards $\{1..5\}, \{1..8\}, \{1..10\}, \{0..10\}$), holding the budget, the model, and the validation shard fixed, so val_bpb is comparable across arms and the only variable is training-data volume. Each arm captured its split at launch (verified); the update count is an outcome.

3. Result: A Steep Data-Scaling Slope

Table 1 is the ablation. val_bpb falls monotonically and steeply as training data grows from 5 to 10 shards, while the step count is essentially constant (≈1970), so the improvement is attributable to data volume (less repetition / later overfitting onset), not to throughput.

Three observations. First, the slope is enormous: halving the data (10→5 shards) costs +0.118 val_bpb — roughly 80 seed-standard-deviations, and larger than the entire +0.082 that 64 rounds of architecture search achieved. Second, the slope is not yet flattening in a way that suggests an architecture floor: −0.029/shard over 5 → 8 and −0.015/shard over 8 → 10 is diminishing but steeply negative, the signature of a model still data-starved, not data-saturated. Third, the lone non-monotone point — the 11th shard (#0) regressing +0.0035 — is a single anomalous or noisy shard added outside the curated 1–10 set; it does not perturb the dominant 5 → 10 trend and, if anything, flags that shard identity (not just count) matters near the margin.

4. The Floor is Data-Limited, and Why That Reframes the Campaign

The +7.9% that six optimized levers reached is a data ceiling. At 10 shards the model runs ∼2 epochs in five minutes — exactly the overfitting boundary that made the budget-scaling test fail — and the data-scaling slope shows it sits well below the loss the same architecture would reach with more tokens. The architecture is not the binding constraint; the corpus is.

This retro-explains the campaign's most striking pattern: every adopted modeling lever was subtractive (FFN 6× → 4×, attention span $2^{10} \to 2^9$) or O(1)-lookup (the n-gram hash memory), and the clearest failures added capacity that must be learned (the catastrophic n-gram→logit head, paper 009). Under ∼2-epoch data starvation the model cannot fully exploit its capacity — there are not enough distinct tokens to train it — so trading capacity for update count (U) pays, and lookup memory (which needs no learning) pays most. This is precisely H9's learning-budget constraint (paper 009 (OPHIS, 2026b)) with the binding budget revealed to be data, not wall-clock.

The subtraction verdict is a knife-edge, and it inverts with data. We can test part of this directly. Table 2 crosses FFN width against data volume. At the 10-shard operating point FFN4× beats FFN6× by +0.004 — the campaign's adopted verdict — but at 8 and 5 shards the verdict reverses: FFN6× wins by −0.019 and −0.021. FFN6×'s advantage shrinks monotonically as data grows (+0.021 → +0.019 → −0.004), crossing over near 9 shards. The mechanism is the epoch-count confound made productive: FFN6× is slower, so it completes fewer epochs, so it overfits less — decisive in the heavily-starved 3–4-epoch regime (5–8 shards) and irrelevant at the mild 2-epoch operating point, where FFN4×'s higher update count wins. So the "subtraction wins" verdict that the campaign adopted is not a property of the architecture; it is a property of the specific ∼2-epoch operating point, and it flips under a small change in data. The same holds for the campaign's other structural subtraction lever: the short-window span (swin4, $2^{10} \to 2^9$) is beaten by the wider span at 5 shards (+0.011) and 8 shards (+0.005), winning only at the 10-shard operating point — so both adopted structural subtractions independently invert with data, by the same fewer-epochs mechanism. This is direct evidence for the inversion

Table 2. FFN width × data volume (fixed budget, fixed val). The adopted verdict (FFN4× best) holds only at the operating point; it inverts under data starvation.

Shards	FFN4×	FFN6×	Winner
5	1.0667	1.0452	FFN6× (−0.021)
8	0.9793	0.9606	FFN6× (−0.019)
10	0.9488	0.9531	FFN4× (+0.004)

Table 3. N-gram memory on/off × data (fixed budget, fixed val). Without the memory, val_bpb is nearly flat across data; with it, val_bpb carries the entire 0.118 swing. The memory's own contribution inverts from a help to a large harm as data shrinks.

Shards	n-gram ON	n-gram OFF	Effect (ON−OFF)
5	1.0667	0.9809	+0.086 (harms)
8	0.9793	0.9776	+0.002 (neutral)
10	0.9488	0.9774	−0.029 (helps)
range	0.118	0.004	—

clause of H10 below. What we cannot reach is the opposite extreme — a data-rich (< 1-epoch) regime where capacity would re-admit for the intended reason (signal to fill it rather than epochs to avoid) — because the node is offline and only shards 0–10 are cached, which is itself the practical punchline: the campaign's dominant remaining lever is more data, and it is the one lever infrastructurally out of reach here.

5. The Data-Limitation Is, Specifically, the N-gram Memory

The inversions of Section 4 concern which width or span is best at a given data volume. A sharper question is how much of the data-sensitivity comes from the dominant lever itself. We disable the n-gram hash memory (table size → 1, a compile-safe proxy that collapses all n-gram keys to a single constant) and re-run the data sweep. The result (Table 3) is the campaign's deepest single finding: the n-gram memory is the sole source of the data-limitation.

Two facts stand out. First, the n-gram-off model is nearly data-insensitive: its val_bpb moves only 0.004 from 5 to 10 shards, against the 0.118 that the full model moves. The transformer-plus-value-embeddings backbone, absent the hash memory, has essentially reached its (mediocre, ∼0.977) data-rich plateau already at 5 shards — it is not data-starved. Second, the memory's own contribution is not a fixed −0.03 "lever value" but a steep function of data: −0.029 (help) at 10 shards, neutral at 8, and +0.086 (a large harm) at

Table 4. N-gram table size × data (fixed budget, fixed val). The optimal table size grows with data: 2^{15} wins when starved, 2^{21} only at the operating point. Data-sensitivity (range) scales with table size.

Table	5 sh	8 sh	10 sh	range
2^0 (off)	0.9809	0.9776	0.9774	0.004
2^{15}	0.9770	0.9696	0.9688	0.008
2^{21}	1.0667	0.9793	0.9488	0.118

5. The mechanism is memorization: the hash memory learns per-context value corrections from training n-gram statistics, which generalize to the validation shard only when the training set is large enough to make those statistics representative. Below ∼8 shards the memorized corrections overfit — the model does 3–4 epochs and the memory encodes shard-specific noise — and the campaign's single best lever becomes its worst.

This subsumes Section 4: the FFN and span inversions are the backbone quietly trading capacity for epochs, but the dominant data-sensitivity, and the entire reason the floor moves with data, is the memorization memory. It also refines H9 and paper 003's hash-table scaling law: the O(1) memory is the largest lever and the most data-fragile, because "needs no learned addressing" (H9) is not the same as "needs no data to generalize." The floor is data-limited because the campaign's best idea is a memorizer, and memorizers are the first thing data starvation breaks.

Memory size is memorization capacity: the optimal table size is data-dependent. If the memory hurts under starvation by over-memorizing, then less memory should hurt less. Table 4 confirms it: a 2^{15} table (64× smaller than the adopted 2^{21}) is the best configuration at 5 and 8 shards — beating both the full memory (catastrophic at 5 shards) and no memory — while 2^{21} wins only at 10 shards, the operating point where paper 003's scaling law was measured. The data-sensitivity scales monotonically with memory size (range 0.004 at 2^0, 0.008 at 2^{15}, 0.118 at 2^{21}), exactly as a memorization-capacity knob should. The optimal memory size therefore grows with data: paper 003's "bigger table is better" scaling law is itself a property of the 10-shard operating point, and at less data the law reverses. This is the memorization account made quantitative — table size is a dial on how much the model memorizes, and how much it should memorize is set by how much data it has.

The transferable lesson is a cheap diagnostic that should run near the start of an autonomous campaign,

not its end. A three-point data-scaling ablation costs three runs and returns the slope $d\,\mathrm{val\_bpb}/d(\mathrm{data})$ at the operating point. A steep slope — as here — is a warning that every subsequent architecture lever is being optimized inside a data-starved regime, where (i) the achievable floor is not the architecture's floor, (ii) capacity levers will read as stale and subtractive levers as wins for reasons that will invert with more data, and (iii) the highest-value action is to acquire data, not to tune. Had we measured this slope at round 1 rather than round 65, the entire lever taxonomy would have carried the caveat that it describes the data-starved regime specifically. We did establish the taxonomy rigorously (papers 001–010) and it is correct for this data budget; the data-scaling result places it.

We state the hypothesis:

> H10 (data-slope diagnostic). In fixed-budget pretraining, the sign and steepness of the data-scaling slope at the operating point determine whether architecture search is measuring architecture or data starvation. A steep slope ($\gg \sigma$ per data-doubling) implies a data-limited floor: subtractive and lookup levers will win, capacity levers will read stale, and both verdicts invert once data ceases to bind. Confidence: 0.8.

Confidence rises to 0.8 from the direct inversion evidence (Table 2): the adopted FFN4×-over-FFN6× verdict genuinely reverses across the accessible data range, confirming that the campaign's lever taxonomy is operating-point-specific rather than architectural. It is held below certainty because the observed inversion is driven by the epoch-count confound (fewer epochs ⇒ less overfitting) rather than by the intended data-rich capacity re-admission, which the offline node blocks; the two mechanisms predict the same sign here but would separate in a $<$ 1-epoch regime, and the diagnostic is validated on one campaign.

6. Conclusion

Asked whether its certified floor was architectural, budget-, or data-imposed, the campaign answers cleanly: data. A fixed-budget, fixed-validation data-scaling ablation shows val_bpb falling $1.0667 \to 0.9793 \to 0.9488$ from 5 to 10 training shards — a -0.118 swing, larger than the entire $+0.082$ of 64 rounds of architecture search, at constant step count. The model trains on 10 of 6543 available shards at $\sim$2 epochs, and its $+7.9\%$ ceiling is a data ceiling: the architecture has substantial headroom that only more tokens can unlock, and the campaign's subtractive/lookup winning levers are the fingerprint of data starvation, not of a good architecture at its limit. A lever-decomposition pins the cause precisely: disabling the n-gram hash memory makes the model nearly data-insensitive (range 0.004 vs 0.118), so the memory — the campaign's single largest lever — is the sole carrier of the data-sensitivity, a memorization mechanism that helps by -0.029 at the operating point but harms by $+0.086$ when starved to 5 shards. The floor is data-limited because the best lever is a memorizer. The dominant remaining lever is more data — infrastructurally out of reach on this offline node — and the portable lesson is to measure the data-scaling slope at the start of an autonomous campaign, so that architecture search knows whether it is measuring the architecture or the size of its corpus.

References

Karpathy, A. nanochat: The best ChatGPT that $100 can buy. 2025. URL https://github.com/karpathy/nanochat.

OPHIS Autonomous Research Agent. Convergence of iterated re-optimization and a mechanism-exhaustion map. Campaign paper 008, AI_papers/, 2026a.

OPHIS Autonomous Research Agent. The basis-limited floor and a learning-budget constraint on new mechanisms. Campaign paper 009, AI_papers/, 2026b.

OPHIS Autonomous Research Agent. Finding a floor and proving it: a 61-round autonomous pretraining campaign. Campaign paper 010, AI_papers/, 2026c.